Prabu Shankar P
ROLL NO 241501142

EXPERIMENT 14

Pro code extension with Catalyst

Aim

Extend the app with a small authenticated Catalyst service and compare it with a Creator custom function.

Procedure

1. Choose a bounded extension: validate an Indian vehicle registration number format (e.g. TN01AB1234) and return a normalized value. It must not authorize services or expose owner data.
2. Create a Catalyst Advanced I/O function (Node.js) with POST /validate-reg and secure it with Catalyst authentication or API Gateway rules.
3. Call it from Creator through a secured connection from a staff-only action; do not call on every keystroke.
4. Handle timeouts and bad JSON separately from 'invalid format'.

```
function validateReg(v){
  if(typeof v!=="string") return {valid:false,reason:"reg must be text"};
  const r=v.replace(/[ -]/g,"").toUpperCase();
  return {reg:r, valid:/^[A-Z]{2}\d{2}[A-Z]{1,3}\d{4}$/.test(r)};
}
```

Adjust the pattern if your region uses other formats (e.g. BH series).

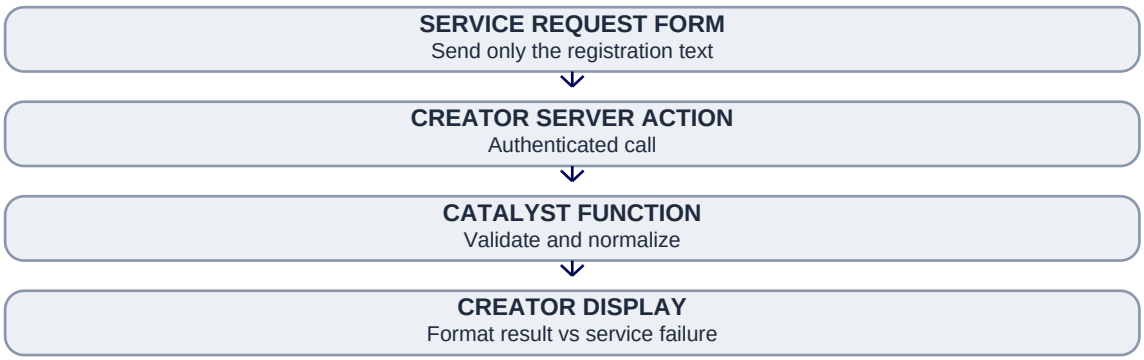
Implementation and verification

Input	Expected
tn01ab1234	Normalized TN01AB1234, valid
TN 01 AB 1234	Normalized, valid
1234	Invalid format
Null or number	Rejected (400)
No credentials	401 or 403

Evidence and testing

E14: function code, interface contract, authenticated Creator call, test results. This is a format check only, not proof the vehicle exists.

Process diagram



Generated explanatory diagram. It is not a screenshot of an executed workflow.

Result

Expected result and verification criteria. Valid and invalid inputs are distinguished from service failures; you can justify when an external service is worth using versus a local Creator function.